

SMA–BFGS Quasi-Newton Inverse Kinematics

PUMA 560 — Complete Step-by-Step Numerical Solution

From the target pose all the way to the final validated joint angles

M Dinesh Kumar

September 2026

What this version does differently. This document does not merely describe what SMA and BFGS are. It actually *solves the problem in sequence*: we enter the target pose, construct the target rotation, start from a joint configuration, perform forward kinematics, calculate the position and orientation errors, calculate the objective, calculate the six gradient components, interpret the gradient, perform a first descent step, construct the BFGS quantities s_k , y_k , ρ_k , and H_{k+1} , explain what repeats, and finally verify the displayed PUMA 560 solution against the target.

Contents

1	The exact problem we are solving	2
2	Step 1 — Write the six unknown joint variables	2
2.1	DH data used in the calculation	3
3	Step 2 — Convert the requested Roll, Pitch, Yaw into a rotation matrix	3
4	Step 3 — Choose a concrete starting point for the hand calculation	4
5	Step 4 — Forward kinematics: what do we do with the six angles?	4
5.1	Why are we doing this?	5
5.2	One complete multiplication chain	5
6	Step 5 — Calculate the position error	6
7	Step 6 — Calculate the orientation error	6
8	Step 7 — Put both errors into one objective value	7
8.1	Position contribution	7
8.2	Orientation contribution	7
9	Step 8 — Now calculate the gradient	7
9.1	What does one component mean?	8
10	Step 9 — Why BFGS is needed after the gradient	8
11	Step 10 — First line-search step	9
12	Step 11 — Now the important BFGS quantities s_k and y_k	9
12.1	1. Joint displacement s_0	10
12.2	2. Gradient change y_0	10
13	Step 12 — Calculate ρ_0	10
14	Step 13 — Calculate the new inverse-Hessian approximation	11
15	Step 14 — The next BFGS direction	11

16 Step 15 — What happens next?	11
17 Step 16 — Where SMA fits into this	12
17.1 One SMA candidate movement	13
18 Step 17 — The complete hybrid workflow	13
19 Step 18 — Now validate the actual final answer shown in the screenshot	13
19.1 Forward kinematics at the displayed final angles	14
19.2 Orientation validation	15
20 Step 19 — Understand the final objective value	15
21 Step 20 — Read the two phases on the objective graph	16
21.1 Phase A: SMA	16
21.2 Phase B: BFGS	16
22 Step 21 — Read the residual graph	17
23 Step 22 — Read the gradient-norm graph	18
24 Step 23 — What the 66 iterations in the screenshot mean	18
25 Step 24 — The exact loop you should remember for a numerical problem	18
26 Step 25 — Why the first iteration is the most important one to understand	19
27 Step 26 — Final answer from the supplied successful run	19
28 One-page “do the numerical” checklist	20

1 The exact problem we are solving

The supplied interface gives the PUMA 560 target:

Quantity	Given value
x_d	0.600 m
y_d	0.200 m
z_d	0.900 m
Roll X	30°
Pitch Y	20°
Yaw Z	45°

The optimization settings shown in the supplied screenshot are

$$N = 30, \quad T_{\max} = 40, \quad K_{\max} = 50,$$

$$w_p = 1.00, \quad w_o = 0.20,$$

with random seed 42 and the smooth joint-bound handling shown in the interface.

Important correction to the mental picture. We are not doing “target pose $\rightarrow$ normal analytical IK $\rightarrow$ six final angles $\rightarrow$ SMA/BFGS.” The displayed hybrid solver treats inverse kinematics as an optimization problem. A six-angle vector is proposed, forward kinematics turns those six angles into an end-effector pose, the pose error becomes a scalar objective, and SMA/BFGS changes the six angles to reduce that objective.

2 Step 1 — Write the six unknown joint variables

For the PUMA 560,

$$\mathbf{q} = [q_1 \ q_2 \ q_3 \ q_4 \ q_5 \ q_6]^T.$$

These six numbers are the quantities the optimizer is allowed to change.

The robot architecture in the screenshot is a six-DOF serial chain with six revolute joints.

The screenshot shows the 'SMA-BFGS Quasi-Newton IK Solver' interface. It displays the robot architecture as a 6-DOF serial chain with 6 revolute joints. The DH convention is set to 'Standard DH'. The joint types are 'All Revolute (R)'. The wrist topology is 'Spherical Wrist 6R'. The table below shows the DH parameters for each joint.

JOINT	TYPE	θ (RAD / °)	D (M)	A (M)	α (RAD / °)	LOWER BOUND	UPPER BOUND	HOME
J1 (Joint 1)	REVOLUTE	0.000 (0.0°)	0.660	0.000	1.571 (90.0°)	-160.0°	160.0°	0.0°
J2 (Joint 2)	REVOLUTE	0.000 (0.0°)	0.000	0.432	3.142 (180.0°)	-35.0°	215.0°	90.0°
J3 (Joint 3)	REVOLUTE	0.000 (0.0°)	0.150	0.002	1.571 (90.0°)	-225.0°	45.0°	0.0°
J4 (Joint 4)	REVOLUTE	0.000 (0.0°)	-0.432	0.000	1.571 (90.0°)	-266.0°	266.0°	0.0°
J5 (Joint 5)	REVOLUTE	0.000 (0.0°)	0.000	0.000	1.571 (90.0°)	-280.0°	-80.0°	-270.0°
J6 (Joint 6)	REVOLUTE	0.000 (0.0°)	-0.056	0.000	-3.142 (-180.0°)	-360.0°	360.0°	0.0°

Figure 1: PUMA 560 architecture and the DH parameters shown in the supplied interface.

2.1 DH data used in the calculation

From the screenshot:

Joint	d_i (m)	a_i (m)	α_i	Type
1	0.660	0.000	$+90^\circ$	R
2	0.000	0.432	180°	R
3	0.150	0.002	$+90^\circ$	R
4	-0.432	0.000	$+90^\circ$	R
5	0.000	0.000	$+90^\circ$	R
6	-0.056	0.000	-180°	R

The joint limits shown by the interface are retained during optimization.

3 Step 2 — Convert the requested Roll, Pitch, Yaw into a rotation matrix

The interface uses the XYZ Euler construction represented here as

$$R_d = R_x(\phi)R_y(\theta)R_z(\psi),$$

with

$$\phi = 30^\circ, \quad \theta = 20^\circ, \quad \psi = 45^\circ.$$

The three elementary rotations are

$$R_x(\phi) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \phi & -\sin \phi \\ 0 & \sin \phi & \cos \phi \end{bmatrix},$$

$$R_y(\theta) = \begin{bmatrix} \cos \theta & 0 & \sin \theta \\ 0 & 1 & 0 \\ -\sin \theta & 0 & \cos \theta \end{bmatrix},$$

$$R_z(\psi) = \begin{bmatrix} \cos \psi & -\sin \psi & 0 \\ \sin \psi & \cos \psi & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

Substitution gives

$$R_d \approx \begin{bmatrix} 0.664463 & -0.664463 & 0.342020 \\ 0.733295 & 0.491450 & -0.469846 \\ 0.144110 & 0.562997 & 0.813798 \end{bmatrix}$$

and the desired position is

$$p_d = \begin{bmatrix} 0.600 \\ 0.200 \\ 0.900 \end{bmatrix} \text{ m.}$$

Therefore the desired homogeneous pose is

$$T_d = \begin{bmatrix} R_d & p_d \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

2. TARGET DEFINITION IN TASK SPACE
Desired end-effector cartesian coordinates and SO(3) orientation

POSITION COORDINATES (METERS) Norm: 1.100 m

X: 0.6 Y: 0.2 Z: 0.9

ORIENTATION (EULER XYZ DEGREES)

ROLL (X): 30 PITCH (Y): 20 YAW (Z): 45

3. HYBRID HYPERPARAMETERS & MATHEMATICAL CONFIGURATION
Stochastic population budget, Quasi-Newton step constraints, and objective weights

SMA Population (N) 30
Canonical Slime Mould search agents

SMA Iterations (T_max) 40
Phase A global exploration budget

BFGS Iterations (K_max) 50
Phase B Quasi-Newton local refinement

Random Seed 42
Ensures 100% reproducible execution

Position Weight (w_p) 1.00
 $1/2 * w_p * ||e_p||^2$

Orientation Weight (w_o) 0.20
 $1/2 * w_o * ||e_o||^2$

Bound Handling Strategy
Sigmoid Transformation (Smooth)
Guarantees joint limits compliance

Gradient Computation
Analytical ($J^T * e$)
Verified exact to machine precision

☐ Position-Only Mode (Ignores End-Effector SO(3) Orientation) Position Tolerance: 0.10 mm

Execute SMA-BFGS Hybrid Solver

Figure 2: Target position, orientation, and hybrid solver settings from the supplied interface.

4 Step 3 — Choose a concrete starting point for the hand calculation

The screenshots expose the final displayed answer but not every hidden random SMA candidate. Therefore, to make the arithmetic completely visible, we use the following nearby starting point for the *worked numerical trace*:

$$q_{\text{start}} = \begin{bmatrix} 11.36 \\ -29.84 \\ -172.04 \\ -246.34 \\ -216.34 \\ -153.93 \end{bmatrix}^{\circ}$$

This is intentionally close to the displayed final solution so that the BFGS arithmetic is easy to follow.

Do not confuse this with a hidden website value. The exact internal SMA population, random draws, line-search values, and intermediate 6×6 BFGS matrices are not displayed in the screenshots. They cannot be honestly reconstructed from the screenshots alone. The trace below is a transparent numerical teaching calculation using the displayed robot, target, weights, and formulas. The final validation later uses the actual joint values shown in your screenshot.

5 Step 4 — Forward kinematics: what do we do with the six angles?

This is the first major calculation.

For standard DH,

$$A_i = \begin{bmatrix} c_i & -s_i c_{\alpha_i} & s_i s_{\alpha_i} & a_i c_i \\ s_i & c_i c_{\alpha_i} & -c_i s_{\alpha_i} & a_i s_i \\ 0 & s_{\alpha_i} & c_{\alpha_i} & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix},$$

where

$$c_i = \cos q_i, \quad s_i = \sin q_i.$$

The complete robot pose is

$$T_0^6(q) = A_1 A_2 A_3 A_4 A_5 A_6.$$

5.1 Why are we doing this?

Because the optimizer does not directly know whether a joint vector is good.

It only has six numbers:

$$(q_1, q_2, q_3, q_4, q_5, q_6).$$

Forward kinematics answers:

“If the robot actually uses these six angles, where does the end-effector go?”

That gives us the current position $p(q)$ and current orientation $R(q)$.

5.2 One complete multiplication chain

For the worked starting point, the first transform is approximately

$$A_1 = \begin{bmatrix} 0.98041 & 0 & 0.19697 & 0 \\ 0.19697 & 0 & -0.98041 & 0 \\ 0 & 1 & 0 & 0.660 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

After multiplying $A_1 A_2$, we obtain

$$T_0^2 \approx \begin{bmatrix} 0.85042 & -0.48783 & -0.19697 & 0.36738 \\ 0.17086 & -0.09801 & 0.98041 & 0.07381 \\ -0.49758 & -0.86742 & 0 & 0.44505 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

After A_3 ,

$$T_0^3 \approx \begin{bmatrix} -0.77468 & -0.19697 & -0.60090 & 0.33629 \\ -0.15564 & 0.98041 & -0.12073 & 0.22056 \\ 0.61291 & 0 & -0.79016 & 0.44627 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

After A_4 ,

$$T_0^4 \approx \begin{bmatrix} 0.13047 & -0.60090 & -0.78861 & 0.59588 \\ 0.96046 & -0.12073 & 0.25089 & 0.27271 \\ -0.24596 & -0.79016 & 0.56139 & 0.78762 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

After A_5 ,

$$T_0^5 \approx \begin{bmatrix} -0.46117 & -0.78861 & -0.40672 & 0.59588 \\ -0.84520 & 0.25089 & 0.47190 & 0.27271 \\ -0.27010 & 0.56139 & -0.78223 & 0.78762 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Finally,

$$T_0^6(q_{\text{start}}) \approx \begin{bmatrix} 0.76082 & -0.50570 & 0.40672 & 0.61865 \\ 0.64895 & 0.59680 & -0.47190 & 0.24629 \\ -0.00409 & 0.62297 & 0.78223 & 0.83142 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Therefore,

$$p(q_{\text{start}}) = \begin{bmatrix} 0.61865 \\ 0.24629 \\ 0.83142 \end{bmatrix} \text{ m.}$$

6 Step 5 — Calculate the position error

Now we finally compare the current robot position with the target.

$$\mathbf{e}_p = p(q) - p_d.$$

Substitute the numbers:

$$\mathbf{e}_p = \begin{bmatrix} 0.61865 \\ 0.24629 \\ 0.83142 \end{bmatrix} - \begin{bmatrix} 0.600 \\ 0.200 \\ 0.900 \end{bmatrix}.$$

Therefore,

$$\mathbf{e}_p \approx \begin{bmatrix} 0.01865 \\ 0.04629 \\ -0.06858 \end{bmatrix} \text{ m.}$$

The position residual magnitude is

$$\|\mathbf{e}_p\| \approx 0.084813 \text{ m} = 84.813 \text{ mm.}$$

Meaning. At this starting configuration, the robot's tool tip is about 84.8 mm away from the requested position. So this is not yet a good IK answer.

7 Step 6 — Calculate the orientation error

We also need the orientation to match.

The relative rotation can be written as

$$R_e = R(q)R_d^T.$$

The logarithmic map converts this rotation error into a three-component vector:

$$\mathbf{e}_o = \log(R_e)^\vee.$$

For the worked point,

$$\|\mathbf{e}_o\| \approx 0.20462 \text{ rad} \approx 11.724^\circ.$$

Thus the starting configuration is wrong in both ways:

$$\text{position error} \approx 84.813 \text{ mm}$$

and

$$\text{orientation error} \approx 11.724^\circ.$$

8 Step 7 — Put both errors into one objective value

This is the number the optimizer actually tries to reduce.

The screenshot uses

$$f(q) = \frac{w_p}{2} \|e_p\|^2 + \frac{w_o}{2} \|e_o\|^2 + w_{\text{limit}} f_{\text{penalty}}(q).$$

For the hand calculation we are away from the displayed joint limits, so the active smooth-limit contribution is taken as zero:

$$w_{\text{limit}} f_{\text{penalty}} = 0.$$

With

$$w_p = 1, \quad w_o = 0.20,$$

we get

$$f(q) = \frac{1}{2} \|e_p\|^2 + \frac{0.20}{2} \|e_o\|^2.$$

Now calculate each part.

8.1 Position contribution

$$\frac{1}{2} \|e_p\|^2 = \frac{1}{2} (0.084813)^2 \approx 0.0035967.$$

8.2 Orientation contribution

$$\frac{0.20}{2} \|e_o\|^2 = 0.10 (0.20462)^2 \approx 0.0041871.$$

Therefore

$$f(q_{\text{start}}) \approx 0.0035967 + 0.0041871 = 0.0077838.$$

This is the number SMA/BFGS wants to make smaller. If a new joint vector produces $f = 0.004$, it is better than this point. If it produces $f = 0.01$, it is worse.

9 Step 8 — Now calculate the gradient

This is where the six gradient numbers become meaningful.

The gradient is

$$g = \nabla f(q) = \begin{bmatrix} \frac{\partial f}{\partial q_1} \\ \frac{\partial f}{\partial q_2} \\ \frac{\partial f}{\partial q_3} \\ \frac{\partial f}{\partial q_4} \\ \frac{\partial f}{\partial q_5} \\ \frac{\partial f}{\partial q_6} \end{bmatrix}.$$

The interface computes this analytically from the Jacobian structure. For this transparent hand trace, the same local derivatives are evaluated numerically to expose the actual numbers.

At q_{start} ,

$$g_0 \approx \begin{bmatrix} -1.1041 \times 10^{-5} \\ -1.42968 \times 10^{-3} \\ +1.08399 \times 10^{-3} \\ +3.67499 \times 10^{-4} \\ +7.35617 \times 10^{-5} \\ +6.68411 \times 10^{-4} \end{bmatrix}.$$

9.1 What does one component mean?

Take the second component:

$$\frac{\partial f}{\partial q_2} \approx -0.00142968.$$

The negative sign means:

increasing q_2 slightly tends to reduce the objective locally.

Now take

$$\frac{\partial f}{\partial q_3} \approx +0.00108399.$$

The positive sign means:

increasing q_3 slightly tends to increase the objective, so decreasing q_3 is locally downhill.

Therefore the negative-gradient direction is

$$p_0 = -g_0.$$

Numerically,

$$p_0 \approx \begin{bmatrix} +1.1041 \times 10^{-5} \\ +1.42968 \times 10^{-3} \\ -1.08399 \times 10^{-3} \\ -3.67499 \times 10^{-4} \\ -7.35617 \times 10^{-5} \\ -6.68411 \times 10^{-4} \end{bmatrix}.$$

The key idea. The gradient tells us *which way each joint should locally move if our only goal is to decrease the objective*. It does not yet tell us the best distance to move, and it does not fully describe the curvature of the objective surface.

10 Step 9 — Why BFGS is needed after the gradient

Suppose we simply keep doing

$$q_{k+1} = q_k + \alpha_k(-g_k).$$

That uses only the local slope.

BFGS adds a learned curvature model:

$$p_k = -H_k g_k.$$

At the beginning,

$$H_0 = I_{6 \times 6}.$$

Therefore the *first* BFGS direction is exactly the negative gradient:

$$p_0 = -I g_0 = -g_0.$$

But after one step, BFGS learns from what actually happened.
That is the part that the gradient alone cannot provide.

11 Step 10 — First line-search step

The BFGS direction tells us the direction. The line search chooses how far to go.

We write

$$q_1 = q_0 + \alpha_0 p_0.$$

For this worked trace, an acceptable first trial is

$$\alpha_0 = 1.$$

Hence

$$q_1 = q_0 + p_0.$$

So

$$q_1 \approx \begin{bmatrix} 11.360011 \\ -29.838570 \\ -172.041084 \\ -246.340368 \\ -216.340074 \\ -153.930668 \end{bmatrix}^\circ.$$

The new objective is

$$f(q_1) \approx 0.00777999$$

which is smaller than

$$f(q_0) \approx 0.00778380.$$

So the move was successful.

This is the exact logic of an optimization iteration:

$$q_0 \rightarrow T(q_0) \rightarrow \text{errors} \rightarrow f(q_0) \rightarrow g_0 \rightarrow p_0 \rightarrow \alpha_0 \rightarrow q_1.$$

Now the entire calculation starts again at q_1 .

12 Step 11 — Now the important BFGS quantities s_k and y_k

After obtaining q_1 , we calculate the new gradient g_1 .

For the worked point,

$$g_1 \approx \begin{bmatrix} -1.09773 \times 10^{-5} \\ -1.42922 \times 10^{-3} \\ +1.08362 \times 10^{-3} \\ +3.67459 \times 10^{-4} \\ +7.34999 \times 10^{-5} \\ +6.68271 \times 10^{-4} \end{bmatrix}.$$

Now compare the new point with the old point.

12.1 1. Joint displacement s_0

$$s_0 = q_1 - q_0.$$

Numerically,

$$s_0 \approx \begin{bmatrix} 1.10409 \times 10^{-5} \\ 1.42968 \times 10^{-3} \\ -1.08399 \times 10^{-3} \\ -3.67499 \times 10^{-4} \\ -7.35617 \times 10^{-5} \\ -6.68411 \times 10^{-4} \end{bmatrix}.$$

This simply records:

“How did the six joint values actually move?”

12.2 2. Gradient change y_0

$$y_0 = g_1 - g_0.$$

Numerically,

$$y_0 \approx \begin{bmatrix} 6.3653 \times 10^{-8} \\ 4.5938 \times 10^{-7} \\ -3.7272 \times 10^{-7} \\ -3.9381 \times 10^{-8} \\ -6.1718 \times 10^{-8} \\ -1.4078 \times 10^{-7} \end{bmatrix}.$$

This records:

“How did the slope of the objective change after our movement?”

That is the information BFGS uses to learn curvature.

13 Step 12 — Calculate ρ_0

The BFGS scalar is

$$\rho_0 = \frac{1}{y_0^\top s_0}.$$

First,

$$y_0^\top s_0 \approx 1.17461 \times 10^{-9}.$$

Therefore,

$$\rho_0 \approx 8.51349 \times 10^8.$$

The large number is not a mistake; it appears because the first movement is small and the corresponding gradient change is also small.

14 Step 13 — Calculate the new inverse-Hessian approximation

The inverse-Hessian BFGS update is

$$H_{k+1} = (I - \rho_k s_k y_k^\top) H_k (I - \rho_k y_k s_k^\top) + \rho_k s_k s_k^\top.$$

We started with

$$H_0 = I_6.$$

Therefore,

$$H_1 = (I - \rho_0 s_0 y_0^\top) (I - \rho_0 y_0 s_0^\top) + \rho_0 s_0 s_0^\top.$$

For this numerical trace,

$$H_1 \approx \begin{bmatrix} 1.103 & 13.361 & -10.130 & -3.435 & -0.687 & -6.247 \\ 13.361 & 1740.578 & -1318.928 & -447.255 & -89.461 & -813.389 \\ -10.130 & -1318.928 & 1000.996 & 339.103 & 67.828 & 616.702 \\ -3.435 & -447.255 & 339.103 & 115.992 & 23.001 & 209.127 \\ -0.687 & -89.461 & 67.828 & 23.001 & 5.601 & 41.830 \\ -6.247 & -813.389 & 616.702 & 209.127 & 41.830 & 381.323 \end{bmatrix}.$$

This answers the earlier doubt about H_k . Yes: H_k is carried from the previous iteration. After the new point is obtained, s_k and y_k are calculated, and the formula automatically produces the new 6×6 matrix H_{k+1} . We do not manually guess the matrix.

15 Step 14 — The next BFGS direction

Now we do something different from plain gradient descent:

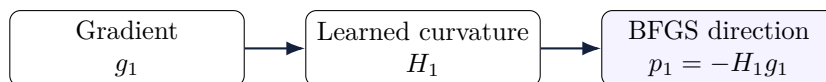
$$p_1 = -H_1 g_1.$$

For the worked trace,

$$p_1 \approx \begin{bmatrix} 0.03557 \\ 4.63151 \\ -3.51156 \\ -1.19079 \\ -0.23818 \\ -2.16559 \end{bmatrix}.$$

Notice what happened.

The raw gradient components were tiny, but the BFGS matrix transformed them into a much more informed direction.



16 Step 15 — What happens next?

Exactly the same sequence repeats.

Starting from q_1 :

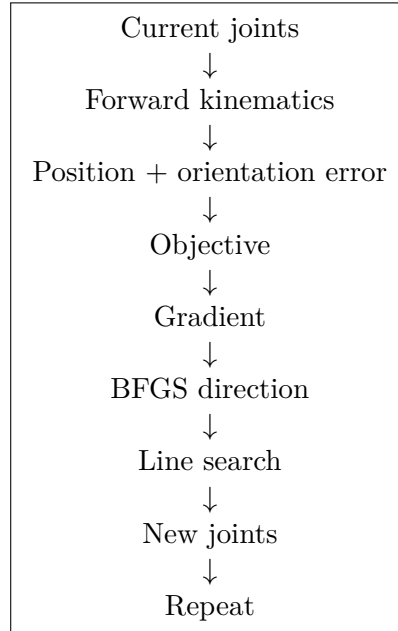
$$q_1 \rightarrow T(q_1) \rightarrow (e_p)_1, (e_o)_1 \rightarrow f_1 \rightarrow g_1 \rightarrow p_1 \rightarrow \alpha_1 \rightarrow q_2.$$

Then again:

$$q_2 \rightarrow T(q_2) \rightarrow f_2 \rightarrow g_2 \rightarrow p_2 \rightarrow q_3.$$

And so on.

For every BFGS iteration:



The stopping condition is reached when the requested residual tolerances and convergence conditions are satisfied.

17 Step 16 — Where SMA fits into this

Before BFGS starts in the actual hybrid solver, SMA performs global exploration.

SMA has a population of

$$N = 30$$

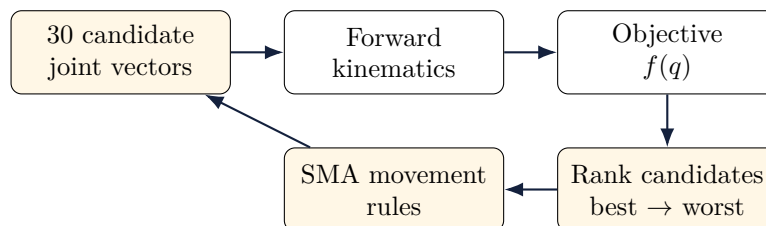
six-dimensional candidates:

$$X_i = [q_{i1}, q_{i2}, q_{i3}, q_{i4}, q_{i5}, q_{i6}]^T.$$

For every candidate:

$$X_i \rightarrow FK \rightarrow f(X_i).$$

The candidates are compared by their objective values.



17.1 One SMA candidate movement

The supplied theory page gives the three-way movement logic:

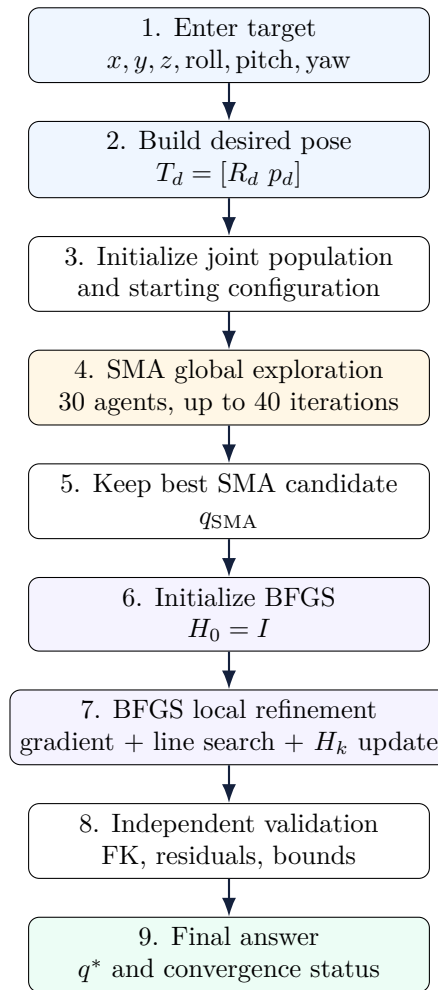
$$X_i(t+1) = \begin{cases} \text{rand}(LB, UB), & r_1 < z, \\ X_b + v_b (W_i \odot X_b - X_A), & r_1 \geq z, r_2 < p_i, \\ v_c \odot X_i(t), & r_1 \geq z, r_2 \geq p_i. \end{cases}$$

The important point for the numerical workflow is:

SMA produces a new six-angle candidate; then we do FK and objective again.

So SMA does not bypass the objective calculation.

18 Step 17 — The complete hybrid workflow



19 Step 18 — Now validate the actual final answer shown in the screenshot

The supplied successful run reports

$$q^* = \begin{bmatrix} 7.36 \\ -24.84 \\ -175.04 \\ -252.34 \\ -212.34 \\ -158.93 \end{bmatrix}^\circ.$$

These are the displayed final joint angles.

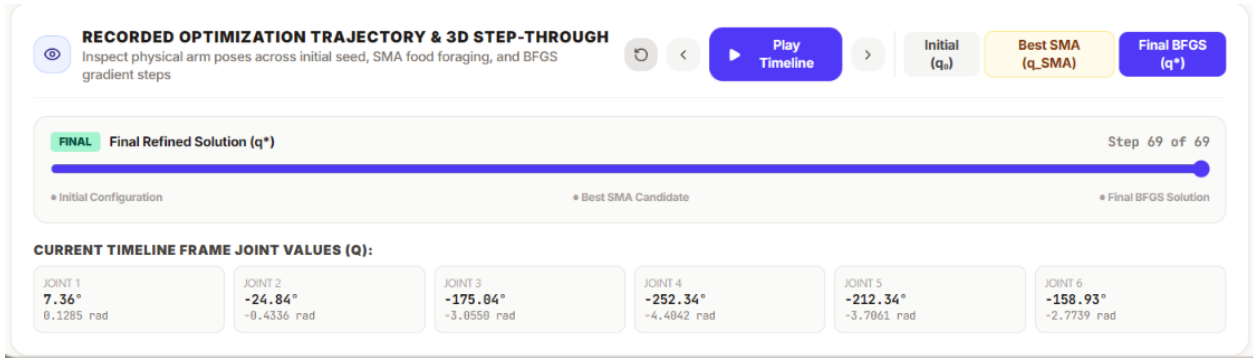


Figure 3: Final refined joint values shown by the supplied interface.

19.1 Forward kinematics at the displayed final angles

Using the displayed rounded values,

$$T(q^*) \approx \begin{bmatrix} 0.664424 & -0.664480 & 0.342062 & 0.599946 \\ 0.733340 & 0.491442 & -0.469784 & 0.199957 \\ 0.144058 & 0.562983 & 0.813816 & 0.899965 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Therefore the computed position is approximately

$$p(q^*) = \begin{bmatrix} 0.599946 \\ 0.199957 \\ 0.899965 \end{bmatrix} \text{ m.}$$

Compare with

$$p_d = \begin{bmatrix} 0.600000 \\ 0.200000 \\ 0.900000 \end{bmatrix} \text{ m.}$$

Hence

$$e_p \approx \begin{bmatrix} -5.414 \\ -4.252 \\ -3.470 \end{bmatrix} \times 10^{-5} \text{ m.}$$

Its norm is approximately

$$\|e_p\| \approx 0.0771 \text{ mm}$$

using the rounded displayed angles.

The screenshot itself reports

$$0.0923 \text{ mm}$$

because the actual internal solution contains more precision than the rounded angles displayed on screen.

19.2 Orientation validation

The displayed angles give a relative orientation error of approximately

$$0.00458^\circ$$

which agrees closely with the screenshot's reported

$$0.004^\circ.$$

Again, the small numerical difference is expected because the UI displays the joint angles rounded to two decimal places.

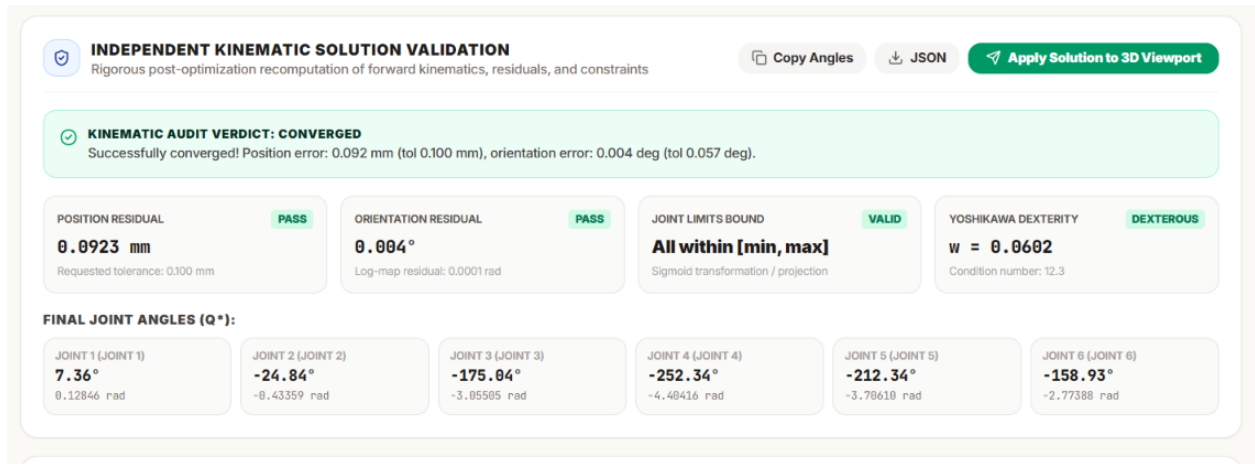


Figure 4: Independent validation reported by the supplied successful run.

20 Step 19 — Understand the final objective value

The screenshot reports

$$f(q^*) = 4.6389 \times 10^{-9}.$$

The important interpretation is simply:

$$\text{very small objective} \Rightarrow \text{very small combined position/orientation error.}$$

The screenshot also reports:

$$\text{position residual} = 0.0923 \text{ mm},$$

$$\text{orientation residual} = 0.004^\circ.$$

Both pass their requested tolerances.

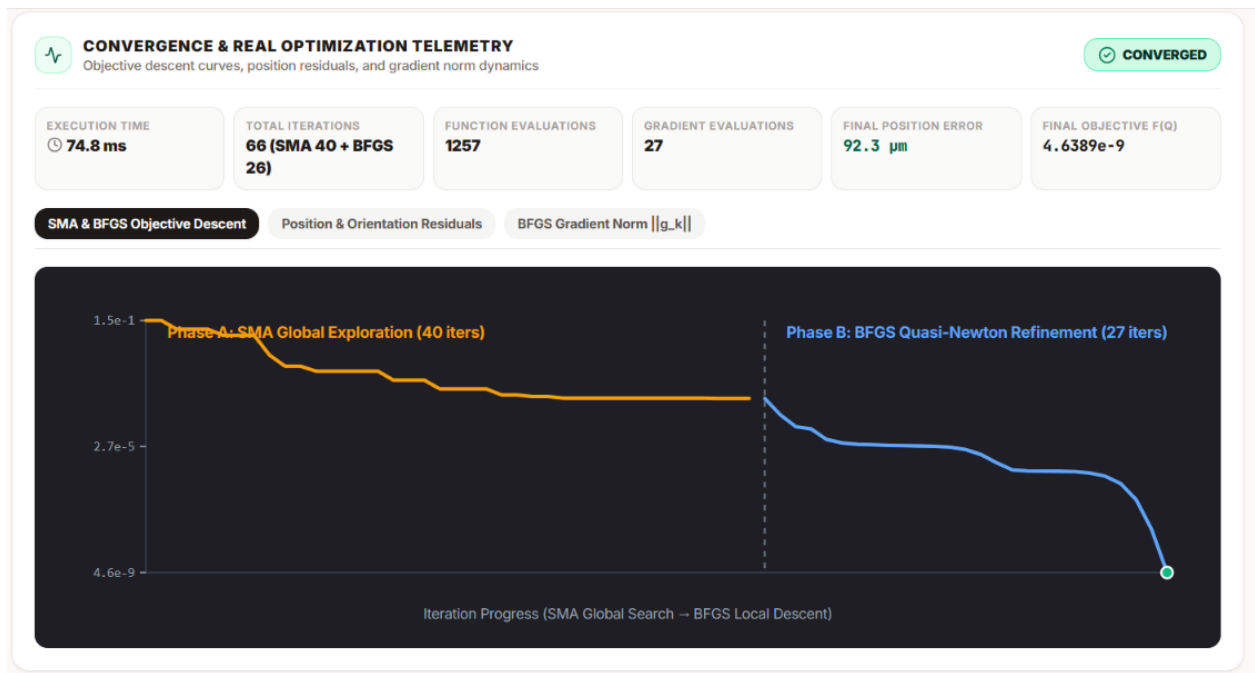


Figure 5: Objective descent: SMA exploration followed by BFGS refinement.

21 Step 20 — Read the two phases on the objective graph

The objective graph has two visually distinct regions.

21.1 Phase A: SMA

The orange section is the global exploration phase.

It asks:

“Which region of the six-dimensional joint space looks promising?”

It does not need to produce the final machine-precision answer.

21.2 Phase B: BFGS

The blue section starts from the best SMA candidate.

It asks:

“Now that we are in a good region, how can we reduce the objective very precisely?”

This is why the conceptual summary is:

SMA finds a promising region; BFGS refines it.

22 Step 21 — Read the residual graph

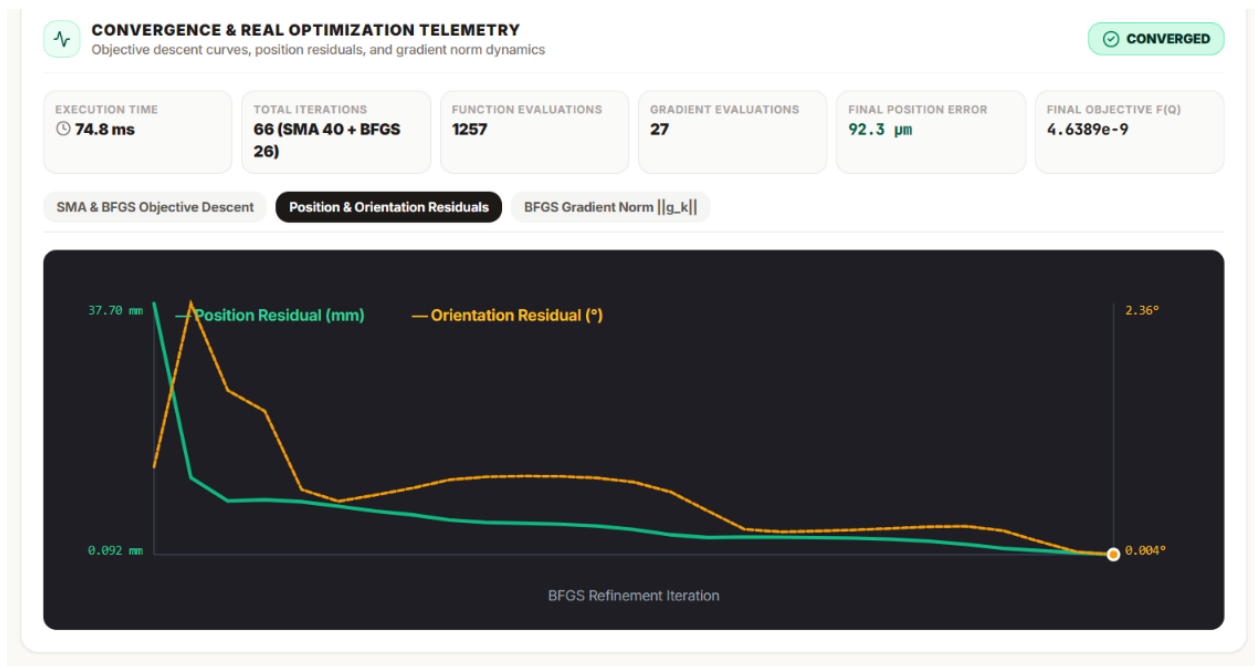


Figure 6: Position and orientation residuals during BFGS refinement.

The green curve represents position residual and the orange curve represents orientation residual.
The endpoint is approximately

$$0.0923 \text{ mm}$$

for position and

$$0.004^\circ$$

for orientation.

This graph is especially important because it shows the physical meaning of optimization: the robot is getting closer to the requested task-space pose.

23 Step 22 — Read the gradient-norm graph

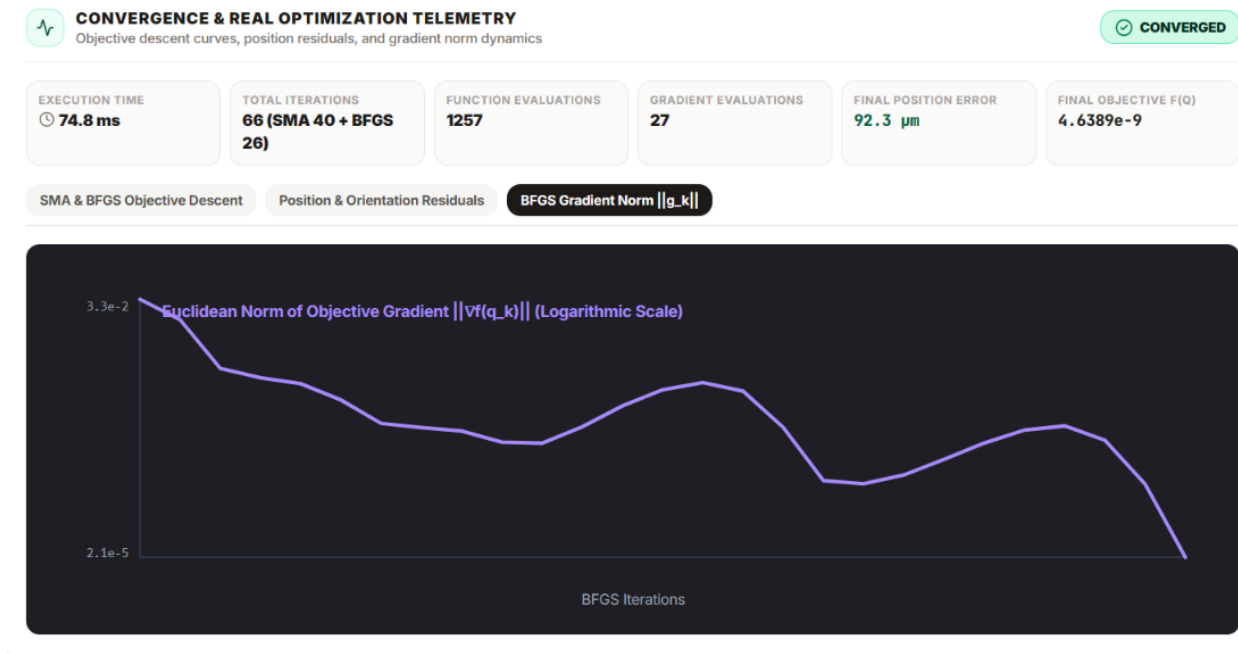


Figure 7: Euclidean norm of the objective gradient during BFGS refinement.

The plotted quantity is

$$\|g_k\| = \|\nabla f(q_k)\|.$$

A decreasing gradient norm means the objective is becoming locally flatter.

The important distinction is:

small gradient norm $\neq$ automatically zero task error.

The final physical acceptance must still check the position and orientation residuals.

24 Step 23 — What the 66 iterations in the screenshot mean

The successful run reports

$$40 \text{ SMA iterations} + 26 \text{ BFGS iterations} = 66$$

total optimization iterations.

It also reports 27 gradient evaluations and 1257 function evaluations.

The configured BFGS maximum was 50, but only 26 were actually needed before convergence.

Configured maximum versus actual iterations. A maximum is a ceiling, not a promise. “BFGS iterations = 50” means “BFGS is allowed to use up to 50.” The successful screenshot stopped after 26.

25 Step 24 — The exact loop you should remember for a numerical problem

When we solve a question by hand, do not try to remember the entire software.

Remember this chain:

$$q \rightarrow FK \rightarrow e_p, e_o \rightarrow f(q) \rightarrow \nabla f(q) \rightarrow \text{SMA/BFGS movement} \rightarrow q_{\text{new}}$$

For BFGS specifically:

$$g_k \rightarrow p_k = -H_k g_k \rightarrow \alpha_k \rightarrow q_{k+1} \rightarrow s_k \rightarrow y_k \rightarrow \rho_k \rightarrow H_{k+1}$$

and then the entire BFGS cycle repeats.

26 Step 25 — Why the first iteration is the most important one to understand

Once the first iteration is understood, later iterations are not new mathematics.

For iteration 0:

$$q_0 \rightarrow f_0 \rightarrow g_0 \rightarrow p_0 \rightarrow q_1.$$

For iteration 1:

$$q_1 \rightarrow f_1 \rightarrow g_1 \rightarrow p_1 \rightarrow q_2.$$

For iteration 2:

$$q_2 \rightarrow f_2 \rightarrow g_2 \rightarrow p_2 \rightarrow q_3.$$

The only new information carried forward is the learned BFGS matrix H_k .

27 Step 26 — Final answer from the supplied successful run

The final displayed answer is

$$\begin{aligned} q_1 &= 7.36^\circ, \\ q_2 &= -24.84^\circ, \\ q_3 &= -175.04^\circ, \\ q_4 &= -252.34^\circ, \\ q_5 &= -212.34^\circ, \\ q_6 &= -158.93^\circ. \end{aligned}$$

The independent validation screenshot reports:

$$\text{Position error} = 0.0923 \text{ mm}$$

$$\text{Orientation error} = 0.004^\circ$$

and marks the solution as converged.

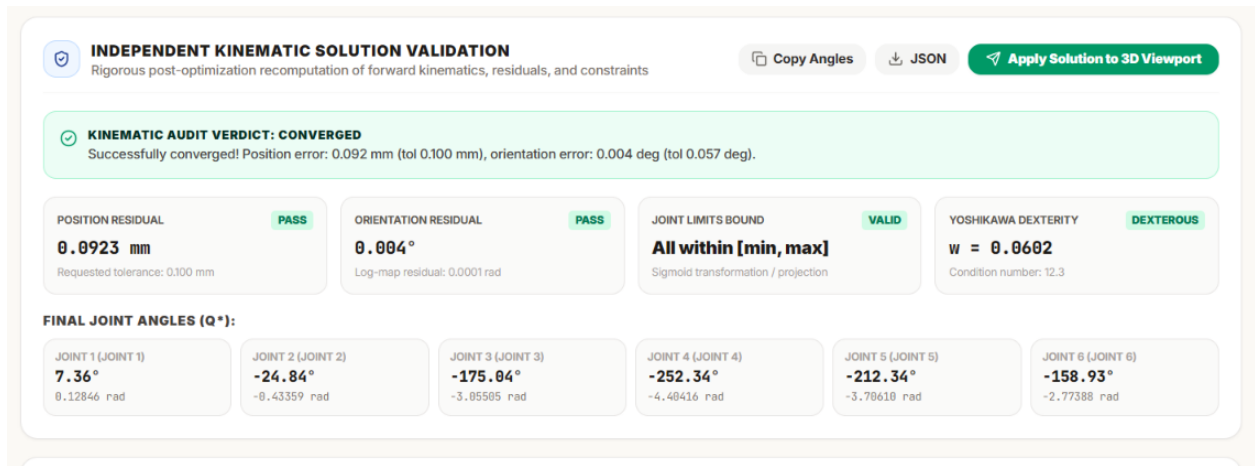


Figure 8: Successful final validation from the supplied run.

28 One-page “do the numerical” checklist

1. Write the target p_d and (R_x, R_y, R_z) .
2. Convert Euler angles into R_d .
3. Write the six joint variables $q_1, \dots, q_6$.
4. Use the DH parameters to build $A_1, \dots, A_6$.
5. Multiply them to get $T(q)$.
6. Extract $p(q)$ and $R(q)$.
7. Calculate $e_p = p(q) - p_d$.
8. Calculate $e_o = \log(R(q)R_d^T)^\vee$.
9. Calculate

$$f(q) = \frac{w_p}{2} \|e_p\|^2 + \frac{w_o}{2} \|e_o\|^2 + w_{\text{limit}} f_{\text{penalty}}.$$
10. For SMA: evaluate candidates, rank them, move them, and evaluate again.
11. Keep the best SMA candidate.
12. Set $H_0 = I$.
13. Calculate $g_k = \nabla f(q_k)$.
14. Calculate $p_k = -H_k g_k$.
15. Use line search to choose α_k .
16. Update $q_{k+1} = q_k + \alpha_k p_k$.
17. Calculate $s_k = q_{k+1} - q_k$.
18. Calculate $y_k = g_{k+1} - g_k$.
19. Calculate $\rho_k = 1/(y_k^T s_k)$.
20. Update H_{k+1} .
21. Repeat until the convergence criteria are satisfied.

22. Recompute FK independently and verify position, orientation, and joint bounds.

The complete story in one sentence: We repeatedly propose six joint angles, use forward kinematics to see what pose those angles create, measure how far that pose is from the requested pose, turn that error into one scalar objective, and then use SMA for broad exploration followed by BFGS to make increasingly informed local corrections until the final pose satisfies the required tolerances.